

PROJECT PROPOSAL

Smart Transit

AI-Powered Multimodal Journey Planner

A Constraint-Aware Route Planning System for Dhaka City

*An Academic Project on Graph Search,
Multimodal Routing, and Constraint-Based Optimization*

Submitted By

Team Merge Conflict

Department of Computer Science and Engineering

United International University

Course: Artificial Intelligence Laboratory (CSE 3812)

Instructor: Ms. Siana Rizwan

September 8, 2026

Abstract

Urban transportation in a densely populated city such as Dhaka involves multiple transportation modes, including walking, buses, and metro rail. In practical situations, travelers often have several constraints beyond simply finding a path from one location to another. For example, a user may have a limited budget, a specific arrival deadline, a preference for less walking, or a desire to minimize the number of transfers.

This project proposes **Smart Transit**, an academic prototype of an AI-powered multimodal journey planner that accepts natural-language travel requests and generates personalized route recommendations. The system will use the Google Gemini API to understand user queries and extract structured travel constraints such as origin, destination, budget, arrival deadline, and preferences. OpenStreetMap Nominatim will be used to resolve textual locations into geographical coordinates.

The core routing engine will operate on a custom multimodal transportation graph representing selected locations and connections in Dhaka. Four algorithms will be combined for different routing purposes: **A* Search** for efficient route searching, **Dijkstra’s Algorithm** for cost-based route optimization, **Breadth-First Search (BFS)** for minimizing transfers or hops, and **Local Search** for improving candidate routes according to user preferences. A constraint and ranking engine will then filter infeasible routes and identify the *Best Match*, *Fastest*, and *Cheapest* options.

The primary objective of this project is not to build a production-grade navigation platform, but to demonstrate how multiple classical search and optimization algorithms can be combined with modern APIs to solve a practical multimodal route-planning problem.

1. Introduction

Finding a route between two locations is a classical graph-search problem. Traditional shortest-path algorithms generally optimize a single objective such as distance, travel time, or cost. However, real-world transportation decisions are usually multi-objective.

For example, consider the following request:

“I want to travel from UIU to Dhaka University within 100 BDT, arrive before 9 AM, and avoid excessive walking.”

A route that is fastest may exceed the user’s budget. A route that is cheapest may require more walking or take longer. Similarly, a route with fewer transfers may not be the fastest or cheapest.

Smart Transit addresses this problem by combining natural-language query understanding with graph-based route search and constraint-aware ranking.

The system is designed as an academic prototype focused on demonstrating algorithmic concepts. Rather than attempting to model the complete transportation infrastructure of Dhaka, the project will use a selected set of important locations, transit stops, and transportation connections to construct a manageable multimodal graph.

2. Problem Statement

Most basic route-planning systems focus on finding one optimal path according to a predefined metric. Such an approach is insufficient when a traveler provides multiple constraints and preferences.

The problem addressed by this project can be formulated as follows:

Given a natural-language journey request containing an origin, destination, budget, arrival deadline, and optional travel preferences, generate multiple feasible multimodal routes and select the routes that best satisfy the user's requirements.

The system must answer questions such as:

- Which route is the fastest?
- Which route is the cheapest?
- Which route requires the fewest transfers?
- Which feasible route best matches the user's preferences?
- Why was a particular route selected?

3. Motivation

Dhaka's transportation environment makes multimodal journey planning particularly relevant. A traveler may need to combine walking, buses, and metro rail in a single journey.

Users also express transportation requirements naturally rather than as formal parameters. Instead of entering separate values into a form, a user may simply write:

"I have 80 taka and need to reach Farmgate before 8:30. I don't want to walk much."

Natural-language processing can transform such input into structured constraints, while graph algorithms can solve the actual routing problem.

Therefore, the project provides an opportunity to integrate:

1. Natural-language understanding,
2. Geographical location resolution,
3. Graph representation,
4. Classical search algorithms,
5. Constraint handling, and
6. Multi-criteria route ranking.

4. Objectives

The main objectives of Smart Transit are:

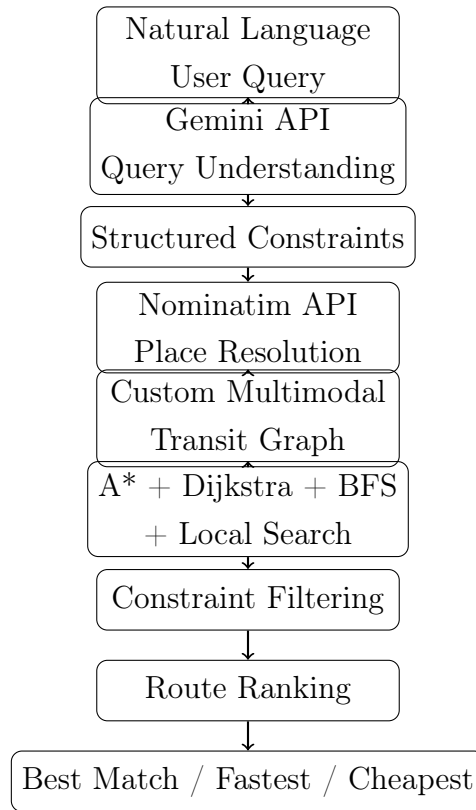
1. To develop a natural-language interface for journey planning.
2. To extract travel constraints such as origin, destination, budget, deadline, and preferences from user queries.
3. To resolve textual place names into geographical coordinates.
4. To construct a simplified multimodal transportation graph for selected areas of Dhaka.
5. To implement and integrate at least four graph-search or optimization algorithms.
6. To use A* Search for efficient path exploration.
7. To use Dijkstra's Algorithm for cost-based route optimization.
8. To use BFS for minimum-hop or minimum-transfer route discovery.
9. To use Local Search for improving candidate routes according to user preferences.
10. To develop a constraint engine that filters routes based on budget and arrival deadline.
11. To rank feasible routes and present Best Match, Fastest, and Cheapest recommendations.
12. To provide a simple explanation of why a particular route was recommended.

5. Proposed System

Smart Transit will consist of several interconnected components.

5.1 High-Level Workflow

The complete workflow is:



6. System Components

6.1 Natural Language Query Processing

The system will accept user requests in either English or Bangla.

Example:

“UIU DU , 100 9 .”

The Google Gemini API will transform the natural-language request into a structured representation.

Example:

```
{
  "origin": "UIU",
  "destination": "Dhaka University",
  "budget": 100,
  "arrival_deadline": "09:00",
  "preferences": {
    "walking": "low",
```

```
    "transfers": "medium"  
  }  
}
```

The Gemini API will only perform query understanding. It will not be responsible for calculating the optimal route.

6.2 6.2 Place Resolution

The OpenStreetMap Nominatim API will be used to resolve textual locations into latitude and longitude coordinates.

For example:

```
"United International University"  
    ->  
Latitude, Longitude
```

The coordinates will then be mapped to the nearest relevant node in the custom transit graph.

6.3 6.3 Multimodal Transit Graph

The routing environment will be represented as a weighted graph.

Each node may represent:

- A landmark,
- A bus stop,
- A metro station, or
- A transfer point.

Each edge may represent:

- Walking,
- Bus travel,
- Metro travel, or
- A transfer between modes.

Each edge will contain simplified attributes such as:

Source
Destination
Mode
Distance
Estimated Time
Fare

Example:

```
{
  "source": "UIU",
  "destination": "Kuril",
  "mode": "walk",
  "time": 8,
  "distance": 0.6,
  "fare": 0
}
```

The graph will be intentionally limited to selected locations in Dhaka to keep the academic implementation manageable.

7. Algorithm Design

The central component of this project is the combination of four algorithms. Each algorithm has a different responsibility. The purpose is not to run four algorithms unnecessarily on the same problem, but to use each algorithm where its characteristics are appropriate.

The four algorithms are:

1. A* Search
2. Dijkstra's Algorithm
3. Breadth-First Search (BFS)
4. Local Search

7.1 A* Search

Purpose

A* Search will be used to find an efficient route between the origin and destination.

A* evaluates nodes using:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the origin to node n ,
- $h(n)$ is the estimated cost from node n to the destination,
- $f(n)$ is the estimated total cost.

For this project, geographical distance can be used as a heuristic for estimating the remaining distance to the destination.

Why A* is Used

A* is suitable because it uses additional information about the destination through a heuristic. Compared with an uninformed search, it can focus the search toward the target instead of exploring the graph uniformly.

Therefore, A* will primarily generate an efficient route candidate for the user's journey.

Example

If the user wants to travel from UIU to Dhaka University, A* will prioritize graph nodes that appear promising based on both the cost already traveled and the estimated remaining distance.

$$f(n) = g(n) + h(n)$$

This makes A* appropriate for the system's general route-search operation.

7.2 Dijkstra's Algorithm

Purpose

Dijkstra's Algorithm will be used to generate a route optimized for a different objective, particularly travel cost.

For a cheapest-route search, the edge weight can be defined as:

$$w(e) = Fare(e)$$

Dijkstra will then find the path with minimum total fare.

Why Dijkstra is Used

Dijkstra's Algorithm is a standard algorithm for finding the shortest path in a weighted graph when edge costs are non-negative.

Its use in this project is different from A*. Instead of primarily searching for a general efficient route, Dijkstra will be used to generate a cost-oriented candidate route.

Therefore:

Dijkstra → Cheapest Route

Example

Suppose there are three possible routes:

Route	Fare
A	80 BDT
B	55 BDT
C	70 BDT

Dijkstra with fare as the edge weight will identify Route B as the cheapest candidate.

7.3 Breadth-First Search (BFS)

Purpose

BFS will be used to identify routes requiring a minimum number of hops or transfers.

BFS is not intended to optimize travel time or fare in this project.

Instead, the transportation graph can be simplified into an unweighted representation where each transition represents one hop or transfer.

For example:

Route A: Walk -> Bus -> Bus -> Metro -> Walk Route B: Walk -> Metro -> Walk
--

Route B contains fewer transportation transitions.

Why BFS is Used

BFS is appropriate for finding the minimum number of edges in an unweighted graph.

This makes it useful for a user preference such as:

“I want to change transportation modes as few times as possible.”

Therefore:

BFS $\rightarrow$ Minimum-Hop / Minimum-Transfer Route

This provides a different route candidate from A* and Dijkstra.

7.4 7.4 Local Search

Purpose

Local Search will be used after the initial candidate routes have been generated.

Unlike A*, Dijkstra, and BFS, Local Search will not primarily discover the initial path from scratch. Instead, it will attempt to improve an existing candidate route by examining nearby alternative routes.

Suppose the initial route is:

UIU $\rightarrow$ Bus Stop A $\rightarrow$ Bus $\rightarrow$ Metro $\rightarrow$ DU

Local Search can examine neighboring alternatives such as:

UIU $\rightarrow$ Bus Stop B $\rightarrow$ Bus $\rightarrow$ Metro $\rightarrow$ DU

UIU $\rightarrow$ Bus Stop A $\rightarrow$ Bus $\rightarrow$ Bus $\rightarrow$ DU

UIU $\rightarrow$ Metro $\rightarrow$ DU

Each neighboring route can be evaluated according to the user's preferences.

A simple objective function can be defined as:

$$Score = w_t T + w_f F + w_w W + w_x X$$

where:

- T = total travel time,
- F = total fare,
- W = walking distance,
- X = number of transfers,
- w_t, w_f, w_w, w_x = preference weights.

Why Local Search is Used

The initial route generated by a shortest-path algorithm may not perfectly match the user's preferences.

For example, a route may be fast but require 1.5 km of walking. A nearby alternative may take only three additional minutes but reduce walking to 400 meters.

Local Search can examine such neighboring alternatives and select an improved candidate.

Therefore:

Local Search $\rightarrow$ Candidate Route Refinement

8. Why the Four Algorithms Are Combined

The algorithms are not being combined simply to increase the number of algorithms used. Each algorithm addresses a different aspect of the routing problem.

Table 1: Role of Algorithms in Smart Transit

Algorithm	Primary Role	Reason for Use
A*	Efficient route search	Uses a heuristic to guide the search toward the destination.
Dijkstra	Cheapest route	Finds the minimum-cost path using fare as the edge weight.
BFS	Minimum transfers/hops	Finds a route with the fewest transitions in an unweighted representation.
Local Search	Route refinement	Improves candidate routes according to walking, time, cost, and transfer preferences.

The overall strategy is therefore:

$$A^* + Dijkstra + BFS + Local Search$$

Each algorithm contributes a different candidate or improvement.

9. Algorithm Blending Pipeline

The algorithmic pipeline of Smart Transit is designed so that each algorithm performs a specific routing task. The algorithms are not used redundantly; instead, their outputs are combined to generate and refine multiple route candidates.

The overall process is as follows:

1. The user provides a natural-language journey request.
2. Gemini extracts the origin, destination, budget, deadline, and travel preferences.
3. Nominatim resolves the origin and destination into geographical coordinates.
4. The coordinates are mapped to the corresponding nodes in the multimodal transit graph.
5. A* Search generates an efficient route candidate by using a heuristic to guide the search toward the destination.
6. Dijkstra's Algorithm generates a cost-optimized route candidate by using fare as the primary edge cost.
7. Breadth-First Search (BFS) generates a route with a minimum number of hops or transportation transfers.
8. The routes generated by the three algorithms are collected into a candidate route pool.
9. Local Search examines neighboring alternatives of the candidate routes and attempts to improve them according to the user's preferences.
10. The Constraint Engine removes routes that violate hard constraints such as budget and arrival deadline.
11. The Ranking Engine evaluates the remaining feasible routes according to travel time, fare, walking distance, and transfers.
12. Finally, the system presents three primary recommendations: Best Match, Fastest, and Cheapest.

The algorithmic relationship can therefore be summarized as:

$$\text{Transit Graph} \rightarrow \begin{cases} A^* & \rightarrow \text{Efficient Route} \\ Dijkstra & \rightarrow \text{Cheapest Route} \\ BFS & \rightarrow \text{Minimum-Transfer Route} \end{cases}$$

The generated routes are then processed as:

Candidate Routes $\rightarrow$ Local Search $\rightarrow$ Constraint Filtering $\rightarrow$ Preference Ranking

Finally, the system produces:

Best Match Fastest Cheapest

10. Constraint Engine

After candidate generation, the system will apply hard constraints provided by the user.

For example:

$$Fare(route) \leq Budget$$

and

$$ArrivalTime(route) \leq Deadline$$

If a route violates the budget:

```
if route.fare > user_budget:
    reject(route)
```

If a route arrives after the deadline:

```
if route.arrival_time > deadline:
    reject(route)
```

This ensures that a route that does not satisfy a user's mandatory requirements is not selected as the Best Match.

11. Route Ranking

After constraint filtering, the remaining routes will be ranked using three primary categories.

11.1 Fastest Route

The fastest route is:

$$R_{fastest} = \arg \min_R TravelTime(R)$$

This route minimizes total estimated travel time.

11.2 11.2 Cheapest Route

The cheapest route is:

$$R_{cheapest} = \arg \min_R Fare(R)$$

This route minimizes the total estimated fare.

11.3 11.3 Best Match Route

The Best Match route will be selected according to user preferences.

A simplified weighted objective can be:

$$Score(R) = w_t T(R) + w_f F(R) + w_w W(R) + w_x X(R)$$

The route with the lowest score among feasible routes will be selected.

For example, if the user strongly prefers less walking, the walking weight w_w will be increased.

Thus:

User Preference $\rightarrow$ Weights $\rightarrow$ Route Score $\rightarrow$ Best Match

12. Example Scenario

Consider the following user query:

“I want to go from UIU to Dhaka University. My budget is 100 BDT, I need to arrive before 9 AM, and I want to minimize walking.”

Gemini may extract:

Origin: UIU
 Destination: Dhaka University
 Budget: 100 BDT
 Deadline: 09:00 AM
 Walking Preference: Low

The algorithms may generate:

Table 2: Example Candidate Routes

Route	Time	Fare	Walking	Transfers
A	45 min	90 BDT	700 m	2
B	61 min	55 BDT	800 m	1
C	52 min	75 BDT	400 m	1

All three routes satisfy the budget and deadline constraints.

Therefore:

- Route A can be selected as **Fastest**.
- Route B can be selected as **Cheapest**.
- Route C can be selected as **Best Match** because it provides a strong balance between time, cost, walking distance, and transfers.

13. API Integration

13.1 13.1 Google Gemini API

Gemini will be used exclusively for natural-language query understanding.

Its responsibilities are:

- Intent understanding,
- Origin extraction,
- Destination extraction,
- Budget extraction,
- Deadline extraction,
- Preference extraction.

The routing algorithms will remain independent of Gemini.

13.2 13.2 OpenStreetMap Nominatim API

Nominatim will be used for geocoding and place resolution.

Its responsibilities are:

- Converting place names into coordinates,
- Resolving common place names,
- Supporting the mapping of user locations to graph nodes.

14. Technology Stack

Table 3: Technology Stack

Component		Technology
Programming Language	Lan-	Python
Natural Language Processing	Language	Google Gemini API
Geocoding		OpenStreetMap Nominatim API
Graph Representation		Python / NetworkX or Custom Graph
Routing Algorithms		A*, Dijkstra, BFS
Optimization		Local Search
Backend		Flask or FastAPI
Frontend		HTML, CSS, JavaScript
Map Visualization		Leaflet.js / OpenStreetMap
Data Storage		JSON or SQLite

15. Data Design

A simplified transportation dataset will be created for selected locations in Dhaka.

Example locations may include:

- United International University,
- Bashundhara,
- Kuril,
- Banani,
- Mohakhali,
- Farmgate,
- Agargaon,
- Shahbag,
- Dhaka University,
- Motijheel.

The exact set of nodes may be adjusted during implementation.

Each connection will contain:

```
{  
  "source": "...",  
  "destination": "...",  
  "mode": "...",  
  "distance": "...",  
  "time": "...",  
  "fare": "..."  
}
```

The data will be kept intentionally small and manageable for the academic prototype.

16. Functional Requirements

The system shall:

1. Accept natural-language travel queries.
2. Support both English and Bangla queries where possible.
3. Extract origin and destination.
4. Extract budget constraints.
5. Extract arrival deadlines.
6. Extract basic user preferences.
7. Resolve locations using Nominatim.
8. Construct or load the transit graph.
9. Execute A* Search.
10. Execute Dijkstra's Algorithm.
11. Execute BFS.
12. Execute Local Search.
13. Filter routes using hard constraints.
14. Rank feasible routes.
15. Display Fastest, Cheapest, and Best Match routes.
16. Provide a basic explanation of the recommendation.

17. Non-Functional Requirements

17.1 Usability

The interface should be simple enough for users to provide travel requirements using natural language.

17.2 Performance

The selected graph will be small enough that route calculations can be completed quickly on a standard computer.

17.3 Maintainability

The system will separate API processing, graph construction, algorithms, constraints, and ranking into independent modules.

17.4 Interpretability

The system should provide understandable reasons for route recommendations rather than presenting only a route identifier.

18. System Limitations

The proposed system is an academic prototype and therefore has several limitations.

- The transportation graph will cover only selected locations rather than the complete Dhaka transportation network.
- Bus routes and estimated travel times may be based on static or manually prepared data.
- The system will not provide production-level real-time navigation.
- Real-time traffic prediction is outside the core scope.
- Weather integration, if implemented, will be treated as an optional enhancement.
- Nominatim may return ambiguous results for certain place names.
- The system will provide estimated route information rather than guaranteed real-world travel times.

These limitations are intentional to keep the project feasible within an academic semester.

19. Evaluation Plan

The system will be evaluated using a set of predefined travel scenarios.

The primary evaluation metrics will include:

1. Total estimated travel time,
2. Total fare,
3. Walking distance,
4. Number of transfers,
5. Algorithm execution time,
6. Constraint satisfaction rate.

Example test cases include:

Table 4: Planned Test Cases

Case	Description
1	Basic origin-to-destination query.
2	Query with a maximum budget.
3	Query with an arrival deadline.
4	Query with a low-walking preference.
5	Query with multiple simultaneous constraints.
6	English natural-language query.
7	Bangla natural-language query.

The output of the algorithms will be compared to verify whether each algorithm fulfills its intended role.

20. Expected Results

The expected outcome is a working academic prototype capable of transforming a natural-language journey request into personalized route recommendations.

For a query such as:

“UIU to DU, under 100 BDT, arrive before 9 AM, less walking.”

the system is expected to produce:

BEST MATCH

Route: Walk -> Bus -> Metro -> Walk

Estimated Time: 52 minutes

Estimated Fare: 75 BDT

Walking: 400 meters

Transfers: 1

FASTEST

Estimated Time: 45 minutes

CHEAPEST

Estimated Fare: 55 BDT

The system should also explain why the Best Match route satisfies the user's requirements.

21. Project Novelty

The novelty of the project does not lie in developing a new graph-search algorithm. Instead, the project demonstrates the practical combination of established algorithms for different aspects of a personalized transportation problem.

The main contribution is the integration of:

**Natural Language Understanding + Geocoding + Multimodal Graph +
Multiple Search Algorithms + Constraint-Based Ranking**

In particular, the system combines:

A → Efficient Search*

Dijkstra → Cost Optimization

BFS → Transfer Minimization

Local Search → Route Refinement

This combination allows the system to provide multiple meaningful route alternatives instead of returning only a single shortest path.

22. Project Scope

The project will focus on the following core features:

1. Natural-language travel query processing.
2. Selected multimodal transportation network of Dhaka.
3. Walking, bus, and metro modes.
4. Four routing/search algorithms.
5. Budget and deadline constraints.
6. Walking and transfer preferences.
7. Route ranking.
8. Basic route explanation.

The project will not attempt to reproduce the full functionality of commercial navigation platforms.

23. Future Work

The prototype can be extended in future work through:

- Real-time traffic integration,
- Real-time bus tracking,
- GTFS-based public transportation schedules,
- Dynamic route re-planning,
- Machine-learning-based travel-time prediction,
- User-history-based personalization,
- Larger-scale Dhaka transportation graphs,
- Real-time transportation disruption detection.

These features are outside the core academic scope of the current project.

24. Project Timeline

Sequence	Task
1	Requirement analysis and project planning
2	Literature review and graph design
3	Transportation data collection
4	Gemini API integration
5	Nominatim integration and location mapping
6	Dijkstra and BFS implementation
7	A* implementation
8	Local Search implementation
9	Constraint and ranking engine
10	Backend and frontend integration
11	Testing and algorithm evaluation
12	Final documentation, presentation, and demonstration

25. Conclusion

Smart Transit proposes an academic prototype for personalized multimodal journey planning in Dhaka. The system combines modern API-based natural-language and location services with classical graph-search and optimization algorithms.

The Google Gemini API will be responsible for understanding natural-language travel requirements, while OpenStreetMap Nominatim will resolve user-provided locations. The actual route computation will be performed by a custom graph-based routing engine.

The central algorithmic contribution is the combination of four algorithms with clearly separated responsibilities. A* Search will be used for efficient route discovery, Dijkstra's Algorithm will generate cost-optimized routes, BFS will identify routes with fewer transfers, and Local Search will refine candidate routes according to user preferences. A constraint and ranking engine will then ensure that recommendations satisfy mandatory requirements such as budget and arrival deadline.

The final system will provide three useful perspectives on a journey: **Fastest**, **Cheapest**, and **Best Match**. Therefore, instead of treating route planning as a simple shortest-path problem, Smart Transit demonstrates how multiple algorithms can be combined to support a more practical and personalized transportation decision.